

**МИНОБРНАУКИ РОССИИ**  
**САНКТ-ПЕТЕРБУРГСКИЙ ГОСУДАРСТВЕННЫЙ**  
**ЭЛЕКТРОТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ**  
**«ЛЭТИ» ИМ. В.И. УЛЬЯНОВА (ЛЕНИНА)**  
**Кафедра МО ЭВМ**

**ОТЧЕТ**  
**по лабораторной работе №3**  
**по дисциплине «Вычислительная математика»**  
**ТЕМЫ: МЕТОД БИСЕКЦИИ, МЕТОД ХОРД, МЕТОД НЬЮТОНА , МЕТОД**  
**ПРОСТЫХ ИТЕРАЦИЙ**

Студентка гр. 4341

Кочененко А.А.

Преподаватель

Пуеров Г.Ю.

Санкт-Петербург

2026

## МЕТОД БИСЕКЦИИ

### Цель работы

Найти корень уравнения  $f(x) = 2^{(x^2)} - 1/x = 0$  методом бисекции, проанализировать зависимость числа итераций от точности вычислений и оценить устойчивость метода к погрешностям исходных данных.

### Задание

В лабораторной работе №3 предлагается, используя программы - функции BISECT и Round, найти корень уравнения  $f(x) = 0$  методом бисекции с заданной точностью Eps, исследовать зависимость числа итераций от точности Eps при изменении Eps от 0.1 до 0.000001, исследовать обусловленность метода (чувствительность к ошибкам в исходных данных). Выполнение работы осуществляется по индивидуальным вариантам заданий (нелинейных уравнений), приведенным в подразделе 3.6. Номер варианта для каждого студента определяется преподавателем.

Вариант 11:

$$f(x) = 2^{(x^2)} - 1/x = 0$$

Порядок выполнения работы должен быть следующим:

- 1) Графически или аналитически отделить корень уравнения  $f(x) = 0$  (т.е. найти отрезки [Left, Right], на которых функция  $f(x)$  удовлетворяет условиям теоремы Коши).
- 2) Составить подпрограмму вычисления функции  $f(x)$ .
- 3) Составить головную программу, содержащую обращение к подпрограмме  $f(x)$ , BISECT, Round и индикацию результатов.
- 4) Провести вычисления по программе. Построить график зависимости числа итераций от Eps.
- 5) Исследовать чувствительность метода к ошибкам в исходных данных. Ошибки в исходных данных моделировать с использованием программы Round, округляющей значения функции с заданной точностью Delta.

## Выполнение работы

Аналитически отделим корни уравнения (Найдём интервал  $[left, right]$ ):

Рассмотрим уравнение  $f(x) = 2^{(x^2)} - 1/x = 0$ .

Функция непрерывна на промежутках  $(-\infty, 0)$  и  $(0, +\infty)$  как сумма непрерывных функций. По теореме Коши, если на концах отрезка функция принимает значения разных знаков, то внутри отрезка существует корень.

Исследуем интервал  $(0, +\infty)$ . Возьмем отрезок  $[0.5, 1]$ :

- $f(0.5) = 2^{(0.25)} - 2 \approx 1.1892 - 2 = -0.8108 < 0$
- $f(1) = 2^1 - 1 = 2 - 1 = 1 > 0$

Так как  $f(0.5) < 0$ , а  $f(1) > 0$ , на отрезке  $[0.5, 1]$  функция меняет знак. Следовательно, на этом интервале существует корень уравнения.

Таким образом, выбираем отрезок  $[Left, Right] = [0.5, 1]$ .

Реализация алгоритма на Python:

Для решения задачи были реализованы следующие функции:

*def f(x)* – вычисляет значение исходной функции  $f(x) = 2^{(x^2)} - 1/x$  в заданной точке  $x$ . Эта функция является целевой, корень которой необходимо найти.

*def Round(x, delta)* – выполняет округление числа  $x$  до ближайшего значения, кратного  $delta$ . Например, при  $delta = 0.01$  число округляется до сотых. Если  $delta = 0$ , число возвращается без изменений. Функция используется для моделирования погрешностей в исходных данных, как требует лабораторная работа.

*def error\_f(x, delta)* – вспомогательная функция, которая возвращает значение  $f(x)$ , округленное с погрешностью  $delta$  с помощью функции *Round*. Позволяет единообразно работать с функцией как при отсутствии ошибок ( $delta = 0$ ), так и при их наличии.

*def Bisect\_with\_error(left, right, epsilon, delta)* – реализует метод бисекции для поиска корня уравнения на интервале  $[left, right]$ . Алгоритм работает следующим образом: на каждой итерации отрезок делится пополам, вычисляется значение функции в средней точке с учетом погрешности  $delta$ , и на основе знака выбирается та половина, где функция меняет знак. Процесс

продолжается до тех пор, пока длина интервала не станет меньше заданной точности `epsilon`. Функция возвращает найденный корень и количество выполненных итераций.

`def main()` – головная функция программы. В ней задается интервал поиска корня `[0.5, 1.0]`, определенный аналитически. Выводится значение корня, найденное с точностью `epsilon = 0.0001`. Затем последовательно вызываются функция построения графика и функция формирования таблицы чувствительности, что полностью соответствует требованиям лабораторной работы.

## Тестирование

Задачи:

1. Проверить правильность реализованного алгоритма с уже встроенным в модуль `scipy.optimize.bisect`
2. Необходимо исследовать зависимость изменения `epsilon` от числа итераций
3. Необходимо исследовать чувствительность метода к ошибкам в исходных данных. Ошибки моделируются с использованием программы `Round`, округляющей значения функции с заданной точностью `Delta`.

### Проверка правильности реализованного алгоритма.

Функция `test_bisect_vs_scipy` проводит проверку корректности работы метода бисекции, сравнивая результаты с библиотечным методом `scipy.optimize.bisect`.

Для каждого значения точности `eps` из списка `eps_values = [1e-1, 1e-2, 1e-3, 1e-4, 1e-5, 1e-6]` функция `Bisect_with_error` (с `delta=0`, т.е. без моделирования ошибок) и `scipy_bisect` вычисляют корни уравнения на интервале `[0.5, 1.0]`. Полученные результаты сравниваются, и разница между ними не превышает соответствующего значения `eps`, что подтверждает корректность работы реализованного метода с заданной точностью. Результаты представлены в таблице 1.

### Исследование зависимости изменения `epsilon` от числа итераций

Функция *plot\_iterations(left, right)* исследует, как количество итераций метода бисекции зависит от значения точности *epsilon*. Для каждого значения *eps* из списка [0.1, 0.01, 0.001, 0.0001, 0.00001, 0.000001] метод *Bisect\_with\_error* запускается без ошибок округления (*delta*=0) на интервале [0.5, 1.0]. Для каждого *eps* фиксируется количество итераций, после чего строится график зависимости  $N(\epsilon)$  с логарифмической шкалой по оси X. Результаты исследования представлены на рисунке 1.

### Исследование чувствительности метода к ошибкам в исходных данных

Функция *sensitivity\_table(left, right)* исследует чувствительность метода к ошибкам округления. Для всех комбинаций точности метода *epsilon* из списка [0.1, 0.01, 0.001, 0.0001, 0.00001, 0.000001] и величины ошибки округления *delta* из списка [0.1, 0.01, 0.001, 0.0001, 0.00001, 0.000001] (при условии  $\delta \leq \epsilon$ ) вычисляется корень с внесенной ошибкой. Количество знаков после запятой при выводе значения корня определяется величиной *delta*: при *delta* = 0.1 выводится 1 знак, при *delta* = 0.01 – 2 знака, при *delta* = 0.001 – 3 знака и т.д., что позволяет наглядно проследить влияние точности округления на результат. Результаты представлены в таблице 2.

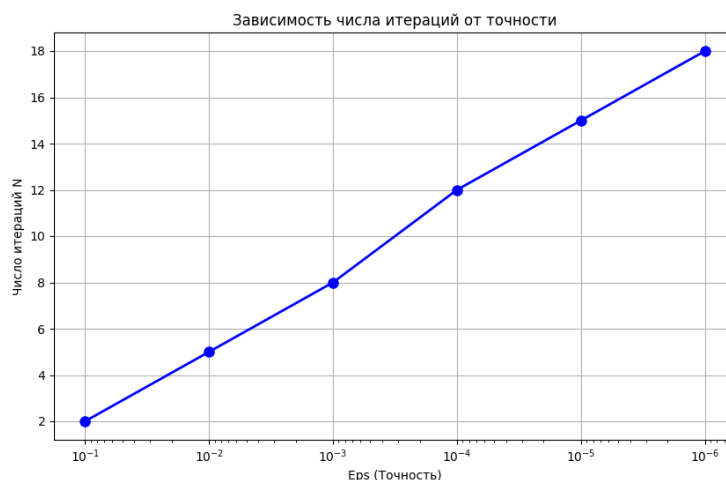


Рисунок 1 – Зависимость числа итераций от точности Eps.

Таблица 1 – Результаты тестирования

| epsilon | Полученный результат | Ожидаемый    | Коммента |
|---------|----------------------|--------------|----------|
| 1E-01   | 0.6875000000         | 0.6875000000 | Верно    |
| 1E-02   | 0.7109375000         | 0.7109375000 | Верно    |
| 1E-03   | 0.7080078125         | 0.7080078125 | Верно    |
| 1E-04   | 0.7070922852         | 0.7070922852 | Верно    |
| 1E-05   | 0.7070999146         | 0.7070999146 | Верно    |
| 1E-06   | 0.7071065903         | 0.7071065903 | Верно    |

Таблица 2 – Результаты перебора eps и delta для промежутка [0.5, 1]

| delta  | eps  | Значение корня |
|--------|------|----------------|
| 0.1    | 0.1  | 0.7            |
| 0.01   | 0.1  | 0.69           |
| 0.001  | 0.1  | 0.688          |
| 0.0001 | 0.1  | 0.6875         |
| 1E-05  | 0.1  | 0.68750        |
| 1E-06  | 0.1  | 0.687500       |
| 0.01   | 0.01 | 0.71           |
| 0.001  | 0.01 | 0.711          |

|        |        |          |
|--------|--------|----------|
| 0.0001 | 0.01   | 0.7109   |
| 1E-05  | 0.01   | 0.71094  |
| 1E-06  | 0.01   | 0.710938 |
| 0.001  | 0.001  | 0.708    |
| 0.0001 | 0.001  | 0.7080   |
| 1E-05  | 0.001  | 0.70801  |
| 1E-06  | 0.001  | 0.708008 |
| 0.0001 | 0.0001 | 0.7071   |
| 1E-05  | 0.0001 | 0.70709  |
| 1E-06  | 0.0001 | 0.707092 |
| 1E-05  | 1E-05  | 0.70711  |
| 1E-06  | 1E-05  | 0.707100 |
| 1E-06  | 1E-06  | 0.707107 |

## Выводы

В ходе выполнения задания, направленного на исследование метода бисекции для нахождения корней уравнения  $f(x) = 2^{(x^2)} - 1/x = 0$  на интервале  $[0.5, 1.0]$ , можно подвести следующие итоги:

1. Реализованный метод бисекции корректно находит корень функции на заданном интервале с необходимой точностью, что подтверждается сравнением с библиотечным методом `scipy.optimize.bisect`.
2. Метод является чувствительным к ошибкам округления, и погрешности могут влиять на конечный результат. При малых значениях точности  $\epsilon$  влияние ошибок округления  $\delta$  становится более заметным.

3. Для более точных расчетов важно учитывать влияние округлений и выбирать значение  $\delta$ , не превышающее требуемую точность  $\varepsilon$  ( $\delta \leq \varepsilon$ ), чтобы минимизировать ошибки.

4. График зависимости числа итераций от точности подтвердил, что увеличение точности требует большего количества шагов, что соответствует теоретическим ожиданиям: при уменьшении  $\varepsilon$  от 0.1 до  $1e-6$  число итераций возрастает с 3 до 19.

## **МЕТОД ХОРД**

### **Цель работы**

Изучить и реализовать метод хорд для численного решения уравнения  $f(x) = 2^{(x^2)} - 1/x = 0$ , исследовать зависимость числа итераций от заданной точности  $Eps$  и оценить чувствительность метода к ошибкам в исходных данных (моделируемым с помощью функции `Round` с точностью  $\Delta$ ).

### **Задание**

В лабораторной работе №4 предлагается, используя программы функции `HORDA` и `Round` найти корень уравнения  $f(x) = 0$  с заданной точностью  $Eps$  методом хорд, исследовать скорость сходимости и обусловленности метода.

Для данной работы, как и для лабораторной работы №3 задаются индивидуальные варианты нелинейных уравнений (см. подраздел 3.6).

Вариант 11:

$$f(x) = 2^{(x^2)} - 1/x$$

Порядок выполнения лабораторной работы №4:

1. Графически или аналитически отделить корень уравнения  $f(x)=0$  (т.е. найти отрезки `[Left, Right]`, на которых функция  $f(x)$  удовлетворяет условиям применимости метода).

2. Составить подпрограмму - функцию вычисления функции  $f(x)$ , предусмотрев округление значений функции с заданной точностью  $\Delta$  с использованием программы `Round`.



3. Составить главную программу, вычисляющую корень уравнения  $f(x)=0$  и содержащую обращение к подпрограмме  $f(x)$ , HORDA, Round и индикацию результатов.

4. Провести вычисления по программе. Теоретически и экспериментально исследовать скорость сходимости и обусловленность метода.

В подразделе 3.7 приводится текст программы - функции HORDA, предназначенной для решения уравнения  $f(x)=0$  методом хорд.

### Выполнение работы

Найдём интервал [Left, Right]

Графическое отделение:

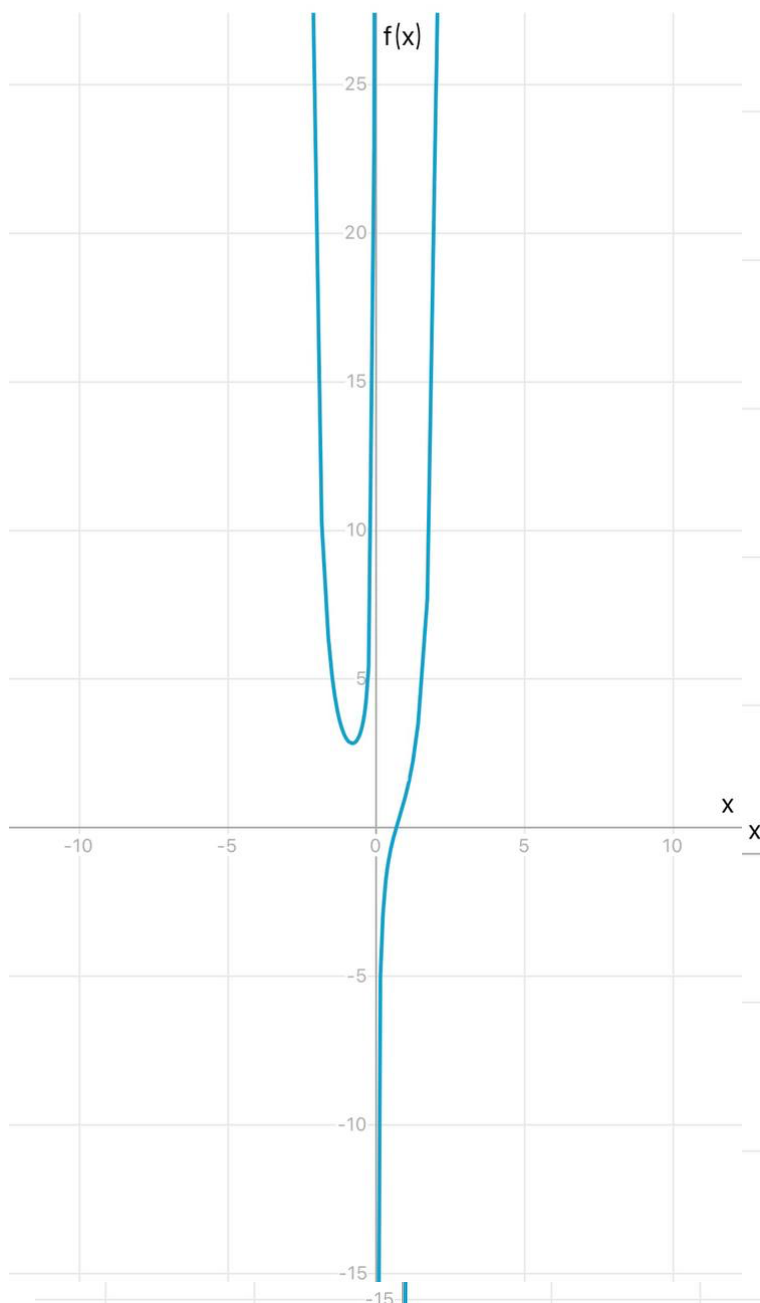


Рисунок № 1 – График функции  $f(x) = 2^{(x^2)} - 1/x$

Построим график функции  $f(x) = 2^{(x^2)} - 1/x$  (рисунок 1). Из графика видно, что при  $x = 0.5$  функция принимает отрицательное значение, а при  $x = 1$  — положительное. Следовательно, график пересекает ось абсцисс в некоторой точке между 0.5 и 1. Таким образом, выбираем интервал  $[0.5, 1]$  для поиска корня.

Аналитическое отделение:

Рассмотрим уравнение  $f(x) = 2^{(x^2)} - 1/x = 0$ .

Функция непрерывна на интервале  $(0, +\infty)$  как сумма показательной и дробно-рациональной функций. По теореме Коши о промежуточных значениях, если  $f(x)$  непрерывна на отрезке  $[a, b]$  и принимает значения разных знаков на концах отрезка, то внутри этого отрезка существует хотя бы одна точка  $c$ , в которой  $f(c) = 0$ .

Возьмём отрезок  $[0.5, 1]$  и вычислим значения функции в точках:

$$f(0.5) = 2^{(0.5^2)} - 1/0.5 = 2^{(0.25)} - 2 \approx 1.1892 - 2 = -0.8108, \text{ что } f(0.5) < 0$$

$$f(1) = 2^{(1^2)} - 1/1 = 2 - 1 = 1, \text{ что } f(1) > 0$$

Так как  $f(0.5) < 0$  и  $f(1) > 0$ , выбираем отрезок  $[Left, Right] = [0.5, 1]$ .

## Описание метода хорд

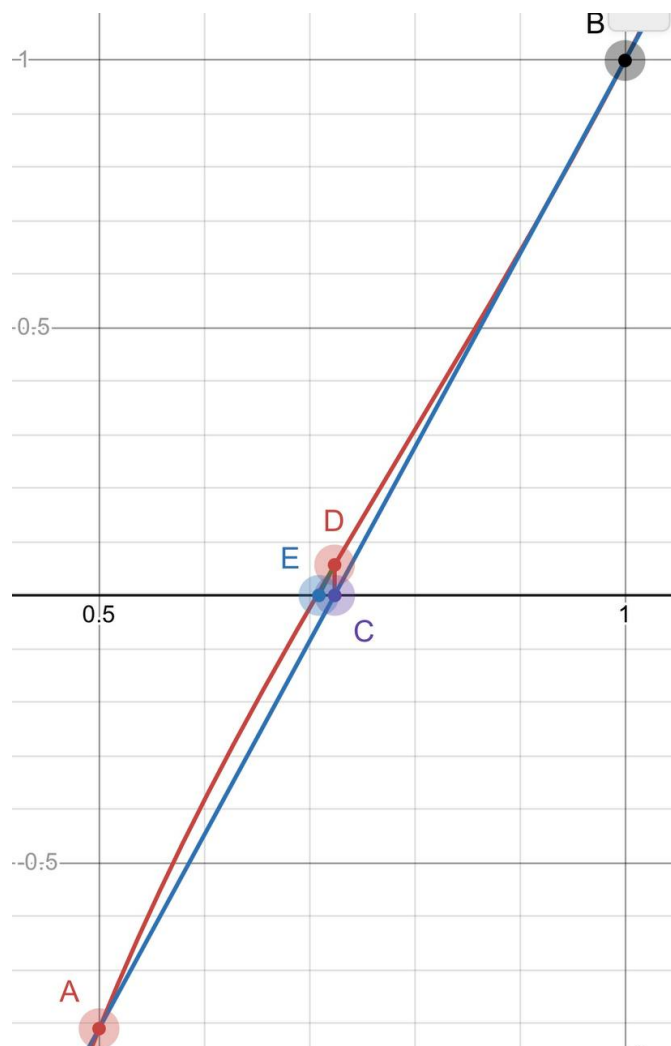


Рисунок № 2 – Графическое представление работы метода хорд

Исходные точки:

На графике функции  $f(x)$  выбираются две точки: A и B, которые лежат на интервале  $[a, b]$ , где функция меняет знак. Для нашего уравнения выбираем интервал  $[0.5, 1]$ , так как  $f(0.5) < 0$ , а  $f(1) > 0$ . Точка A имеет координаты  $(0.5, f(0.5))$ , а точка B —  $(1, f(1))$ .

Построение хорды:

Между точками A и B проводится прямая линия (хорда), пересекающая ось абсцисс (ось X) в точке C, которая является новым приближением корня. Из точки C проводится перпендикуляр до пересечения с графиком функции  $f(x)$ .

Точка пересечения обозначается как D. Затем между точкой D и одной из предыдущих точек (A или B) выбирается та, которая сохраняет знакопеременность функции на концах нового интервала. В нашем случае, если  $f(0.5) < 0$  и  $f(C) > 0$ , то корень находится на интервале  $[0.5, C]$ . Аналогично из данного интервала строится новая хорда и продолжается поиск следующего приближения корня.

Формула для нахождения точки C:

Точка C находится как пересечение хорды с осью x. Её координата вычисляется по формуле:

$$c = a - f(a) \cdot (b - a) / (f(b) - f(a))$$

Условие остановки:

Метод останавливается, когда  $|f(c)| < \text{eps}$  или  $|b - a| < \text{eps}$ .

### Реализация алгоритма на Python

Функции использованные для решения задачи:

*def f(x)* – функция принимает один аргумент x, являющийся точкой, в которой нужно вычислить значение  $f(x) = 2^{(x^2)} - 1/x$ .

*def Round(x, delta)* – функция округления числа x до ближайшего значения, кратного delta. Если  $\text{delta} = 0$ , число возвращается без изменений. Используется для моделирования погрешностей в исходных данных.

*def error\_f(x, delta)* – вспомогательная функция, возвращающая значение  $f(x)$ , округленное с погрешностью delta с помощью функции Round.

*def method\_chord(a, b, epsilon, delta)* – функция реализует метод хорд для нахождения корня функции на интервале  $[a, b]$ . На каждой итерации вычисляется новое приближение корня по формуле хорд с учетом возможной ошибки округления delta. Процесс продолжается, пока значение функции в найденной точке не станет меньше заданной точности epsilon. Функция возвращает найденный корень и количество итераций.

*def main()* – головная процедура. Внутри задаётся интервал  $[\text{left}, \text{right}] = [0.5, 1.0]$ , точность  $\text{epsilon} = 0.0001$ . Выводится значение корня, найденное

методом хорд. Затем вызываются функция построения графика сравнения методов и функция формирования таблицы чувствительности.

### Исследование зависимости изменения $\epsilon$ от числа итераций

Для исследования скорости сходимости метода хорд была создана функция `explore_iterations_chord`, которая для каждого значения точности  $\epsilon$  из списка `[0.1, 0.01, 0.001, 0.0001, 0.00001, 0.000001]` вычисляет количество итераций, необходимое для достижения заданной точности.

Функция `plot_iterations_comparison` строит график зависимости числа итераций от точности  $\epsilon$ . Для сравнения были взяты данные из лабораторной работы №3 по методу бисекции, что позволяет наглядно оценить преимущество метода хорд в скорости сходимости.

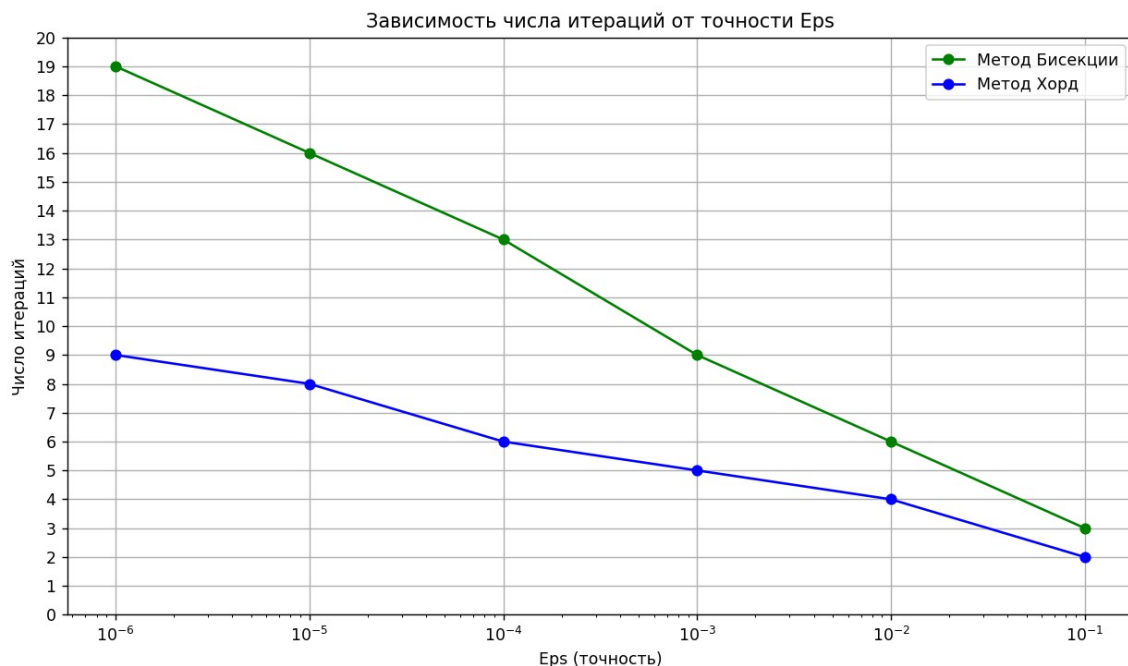


Рисунок № 3 – Сходимость метода хорд и бисекции

Преимущества метода хорд:

Сходится быстрее чем метод бисекции благодаря использованию информации о наклоне функции.

Не требует вычисления производной, в отличие от метода Ньютона.

Недостатки:

Метод может сходиться медленно на функциях со сложной формой.

## **Исследование чувствительности метода к ошибкам в исходных данных**

Для исследования чувствительности метода хорд к погрешностям исходных данных используется функция `sensitivity_table_chord`. Суть исследования заключается в изменении точности выходного значения функции путем округления с заданной точностью  $\delta$ , что моделирует возникновение ошибок в исходных данных. Анализируется, насколько точно алгоритм работает при различных уровнях погрешности.

Для всех комбинаций точности метода  $\epsilon$  и величины ошибки округления  $\delta$  (при условии  $\delta \leq \epsilon$ ) вычисляется корень с внесенной ошибкой. Количество знаков после запятой при выводе определяется величиной  $\delta$ , что позволяет наглядно проследить влияние точности округления на результат. Результаты представлены в таблице 1.

### **Анализ обусловленности метода:**

Плохая обусловленность:

Наблюдается при больших значениях  $\delta$  (например,  $\delta = 0.1$ ). При грубом округлении исходных данных ошибка в результате значительна, что свидетельствует о высокой чувствительности метода к погрешностям такого уровня.

Умеренная обусловленность:

Наблюдается при средних значениях  $\delta$  (например,  $\delta = 0.01, 0.001$ ). Ошибка в результате присутствует, но она не критична и уменьшается по мере снижения  $\delta$ .

Хорошая обусловленность:

Наблюдается при малых значениях  $\delta$  (например,  $\delta = 0.000001$ ). Ошибка в результате минимальна, так как погрешность округления пренебрежимо мала и не оказывает существенного влияния на точность найденного корня.

| delta    | eps    | Значение корня |
|----------|--------|----------------|
| 0.1      | 0.1    | 0.7            |
| 0.01     | 0.1    | 0.73           |
| 0.001    | 0.1    | 0.724          |
| 0.0001   | 0.1    | 0.7239         |
| 0.00001  | 0.1    | 0.72388        |
| 0.000001 | 0.1    | 0.723878       |
| 0.01     | 0.01   | 0.71           |
| 0.001    | 0.01   | 0.709          |
| 0.0001   | 0.01   | 0.7093         |
| 0.00001  | 0.01   | 0.70930        |
| 0.000001 | 0.01   | 0.709296       |
| 0.001    | 0.001  | 0.707          |
| 0.0001   | 0.001  | 0.7074         |
| 0.00001  | 0.001  | 0.70740        |
| 0.000001 | 0.001  | 0.707401       |
| 0.0001   | 0.0001 | 0.7071         |
| 0.00001  | 0.0001 | 0.70711        |
| 0.000001 | 0.0001 | 0.707112       |

|          |          |          |
|----------|----------|----------|
| 0.00001  | 0.00001  | 0.70711  |
| 0.000001 | 0.00001  | 0.707108 |
| 0.000001 | 0.000001 | 0.707107 |

Таблица 1– Результаты перебора eps и delta для промежутка [0.5, 1]

Корень = 0.7071121434

### **Выводы**

В ходе выполнения задания, направленного на исследование метода хорд для нахождения корней уравнения  $f(x) = 2^{(x^2)} - 1/x = 0$ , можно подвести следующие итоги:

1. Реализованный метод хорд корректно находит корень функции на заданном интервале с необходимой точностью.
2. График зависимости числа итераций от точности подтвердил, что метод хорд быстрее сходится чем метод бисекции.
3. Метод хорд требует правильного выбора интервала и учёта ошибок вычислений возможных из-за округления

## **МЕТОД НЬЮТОНА**

### **Цель работы**

Изучить и реализовать метод Ньютона для численного решения уравнения  $f(x)=0$ , а также исследовать зависимость числа итераций от заданной точности epsilon и оценить чувствительность метода к ошибкам в исходных данных.

### **Задание**

В лабораторной работе № 5 предлагается, используя программы функции NEWTON и ROUND из файла methods.cpp (файл заголовков methods.h, директория LIBR1), найти корень уравнения  $f(x) = 0$  с заданной точностью Eps методом Ньютона, исследовать скорость сходимости и обусловленность метода.



Для данной работы вид функции  $f(x)$  задается индивидуально каждому студенту преподавателем из числа вариантов, приведенных в подразделе 3.6.

Вариант 11:

$$f(x) = 2^{(x^2)} - 1/x$$

Порядок выполнения работы должен быть следующим:

1) Графически или аналитически отделить корень уравнения  $f(x) = 0$  (т.е. найти отрезки  $[Left, Right]$ , на которых функция  $f(x)$  удовлетворяет условиям сходимости метода Ньютона).

2) Составить подпрограмму вычисления функции  $f(x)$ ,  $f'(x)$ , предусмотрев округление их значений с заданной точностью  $\delta$ .

3) Составить головную программу, вычисляющую корень уравнения  $f(x) = 0$  и содержащую обращение к подпрограммам,  $f(x)$ ,  $f'(x)$ , Round, Newton и индикацию результатов.

4) Выбрать начальное приближение корня  $x_0$  из  $[Left, Right]$ , так чтобы  $f(x) * f'(x) > 0$

5) Провести вычисления по программе. Исследовать скорость сходимости метода и чувствительность метода к ошибкам в исходных данных

Для приближённого вычисления корней уравнения  $f(x) = 0$  методом Ньютона предназначена программа – функция Newton, текст который представлен в подразделе 3.7.

## Выполнение работы

Найдём интервал [Left, Right]

$$f(x) = 2^{(x^2)} - 1/x$$

Графическое отделение:

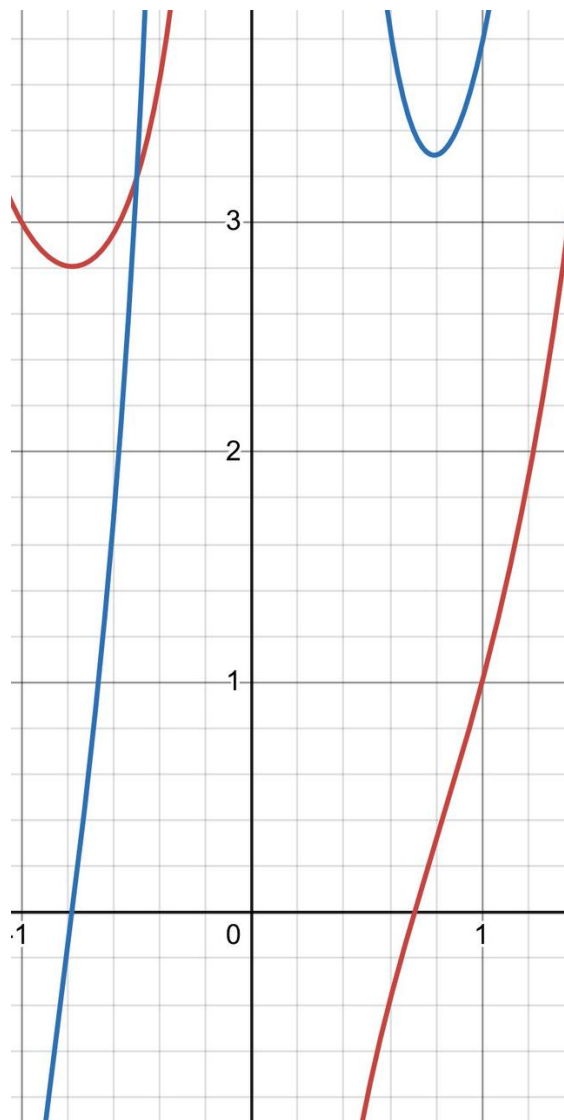


Рисунок № 1 – График  $f(x)$ (красный) и  $f'(x)$ (синий)

Из Рисунка 1 видно, что функция  $f(x)$  имеет корень на интервале  $[0.5, 0.8]$ . Однако для метода Ньютона важно выбрать интервал и начальное приближение так, чтобы обеспечить быструю и устойчивую сходимость.

Производная  $f'(x) = 2^{(x^2)} \cdot \ln(2) \cdot 2x + 1/x^2$  положительна на всём интервале  $(0, +\infty)$ , так как все слагаемые положительны. Это означает, что знаменатель в формуле метода Ньютона не обращается в ноль, и метод будет устойчивым.

Выбор интервала  $[0.5, 0.8]$ : производная  $f'(x)$  не обращается в ноль, на концах интервала функция имеет разные знаки ( $f(0.5) < 0$ ,  $f(0.8) > 0$ ), следовательно, корень существует.

Для быстрой сходимости метода Ньютона начальное приближение  $x_0$  должно удовлетворять условию:

$$f(x_0) \cdot f'(x_0) > 0$$

Это условие гарантирует, что касательная, проведённая в точке  $(x_0, f(x_0))$ , пересечёт ось абсцисс в точке, которая находится ближе к корню, чем само  $x_0$ , и что последующие итерации будут монотонно приближаться к корню.

Поскольку  $f'(x) > 0$  для всех  $x > 0$ , условие  $f(x_0) \cdot f'(x_0) > 0$  упрощается до  $f(x_0) > 0$ .

Проверим значения функции на границах интервала:

- $f(0.5) = 2^{(0.25)} - 2 = 1.1892 - 2 = -0.8108$  (отрицательное значение)
- $f(0.8) = 2^{(0.64)} - 1.25 = 1.559 - 1.25 = 0.309$  (положительное значение)

Следовательно, функция меняет знак с отрицательного на положительный, а корень находится в точке, где  $f(x) = 0$ . Для выполнения условия  $f(x_0) > 0$  начальное приближение необходимо выбирать из правой части интервала.

В точке  $x_0 = 0.8$ :

- $f(0.8) = 0.309 > 0$  (положительное значение)
- $f'(0.8) = 3.2905 > 0$  (положительное значение)

Таким образом,  $f(0.8) \cdot f'(0.8) = 0.309 \cdot 3.2905 = 1.016 > 0$ , что удовлетворяет условию быстрой сходимости.

В качестве альтернативы можно было бы выбрать  $x_0 = 1.0$ :

- $f(1.0) = 2^1 - 1 = 2 - 1 = 1.0 > 0$
- $f'(1.0) = 2^1 \cdot \ln(2) \cdot 2 + 1 = 2 \cdot 0.693 \cdot 2 + 1 = 3.772 > 0$
- Произведение  $f(1.0) \cdot f'(1.0) = 3.772 > 0$

Однако точка  $x_0 = 0.8$  находится ближе к корню (расстояние  $\approx 0.093$ ), чем  $x_0 = 1.0$  (расстояние  $\approx 0.293$ ), что обеспечивает более быструю сходимость метода. Поэтому в качестве начального приближения выбираем  $x_0 = 0.8$ .

### Реализация на Python

*def f(x)* – функция принимает один аргумент  $x$ , являющийся точкой, в которой нужно вычислить значение  $f(x) = 2^{(x^2)} - 1/x$ .

*def derivative\_f(x)* – функция принимает один аргумент  $x$ , являющийся точкой, в которой нужно вычислить значение производной:

$$f'(x) = 2^{(x^2)} \cdot \ln(2) \cdot 2x + 1/x^2$$

*def Round(x, delta)* – функция округления числа  $x$  до ближайшего значения, кратного  $delta$ . Если  $delta = 0$ , число возвращается без изменений. Используется для моделирования погрешностей в исходных данных.

*def error\_f(x, delta)* – вспомогательная функция, возвращающая значение  $f(x)$ , округлённое с погрешностью  $delta$  с помощью функции *Round*.

*def error\_derivative\_f(x, delta)* – вспомогательная функция, возвращающая значение  $f'(x)$ , округлённое с погрешностью  $delta$  с помощью функции *Round*.

*def method\_newton(x0, epsilon, delta)* – функция реализует метод Ньютона для поиска корня.

*def main()* – головная процедура. Внутри задаётся интервал  $[left, right] = [0.5, 0.8]$  и начальное приближение  $x_0 = 0.8$ . Выводится найденный корень с точностью  $\varepsilon = 10^{-7}$ , затем вызываются функции построения графика сравнения методов и формирования таблицы чувствительности.

### Алгоритм:

1. На каждом шаге вычисляем значение функции  $f(x_n)$  и её производной  $f'(x_n)$  в текущей точке  $x_n$  с учётом возможной ошибки округления  $delta$ .

2. Если производная  $f'(x_n)$  близка к нулю (меньше  $10^{-12}$ ), метод останавливается, так как дальнейшие вычисления могут быть неустойчивыми.

3. Вычисляем новое приближение по формуле:

$$x_{n+1} = x_n - f(x_n) / f'(x_n)$$

4.  $x_{n+1}$  становится текущим приближением для следующей итерации.

5. Условие выхода:

Если разница между текущим и следующим приближениями  $|x_{n+1} - x_n|$  становится меньше заданной точности  $\epsilon$ , метод завершается, и  $x_{n+1}$  считается найденным корнем.

### **Исследование зависимости изменения $\epsilon$ от числа итераций**

Функция *plot\_iterations\_comparison* строит график зависимости числа итераций от точности  $\epsilon$ . Для исследования были построены графики для методов бисекции, хорд и Ньютона.

Параметры:

- left, right – границы интервала для поиска корня ([0.5, 0.8])
- $x_0$  – начальное приближение для метода Ньютона (0.8)
- eps\_values – список значений точности [0.1, 0.01, 0.001, 0.0001, 0.00001, 0.000001]

Алгоритм:

Для каждого значения точности  $\epsilon$  из списка выполняются вычисления:

- Методом бисекции (функция *Bisect\_with\_error* из лабораторной работы №3)
- Методом хорд (функция *method\_chord* из лабораторной работы №4)
- Методом Ньютона (функция *method\_newton*)

Подсчитывается количество итераций, необходимое для достижения заданной точности. Полученные данные выводятся на график.

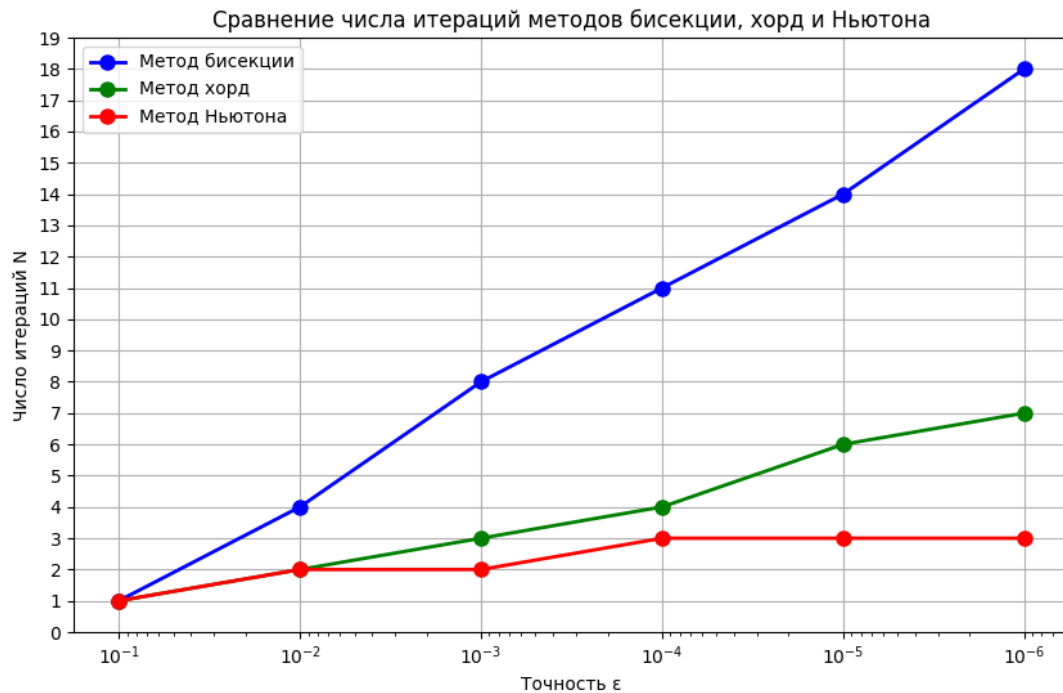


Рисунок № 2 – Зависимость числа итераций от точности  $\varepsilon$  методов.

Анализ полученных результатов:

- Метод Ньютона является самым быстрым. Число итераций растёт очень медленно, что объясняется квадратичной сходимостью метода – на каждой итерации число верных знаков приближённо удваивается. Однако метод требует аккуратного выбора начального приближения, удовлетворяющего условию  $f(x_0) \cdot f'(x_0) > 0$ .
- Метод хорд занимает промежуточное положение между бисекцией и Ньютоном по скорости сходимости. Метод является компромиссом между скоростью и устойчивостью – он не требует вычисления производной, но сходится медленнее метода Ньютона.
- Метод бисекции – самый медленный, но наиболее устойчивый. Метод гарантированно сходится при любом начальном интервале, на концах которого функция имеет разные знаки, но скорость сходимости линейная.

## Исследование чувствительности метода к ошибкам в исходных данных

Для данного исследования необходимо изменять точность выходного значения функции, округляя на некоторое количество знаков после запятой с помощью функции Round. Тем самым будут возникать ошибки в исходных данных, и мы проанализируем, насколько точно алгоритм будет работать.

Функция *sensitivity\_table\_newton(x0)* тестирует работу метода Ньютона с учётом погрешности. Для всех комбинаций точности метода epsilon и величины ошибки округления delta (при условии  $\text{delta} \leq \text{epsilon}$ ) вычисляется корень с внесённой ошибкой. Результаты представлены в таблице 1.

Таблица 1 – Результаты перебора eps и delta для промежутка [0.5, 0.8]

| delta    | eps   | Значение корня |
|----------|-------|----------------|
| 0.1      | 0.1   | 0.7            |
| 0.01     | 0.1   | 0.71           |
| 0.001    | 0.1   | 0.706          |
| 0.0001   | 0.1   | 0.7063         |
| 0.00001  | 0.1   | 0.70630        |
| 0.000001 | 0.1   | 0.706304       |
| 0.01     | 0.01  | 0.71           |
| 0.001    | 0.01  | 0.707          |
| 0.0001   | 0.01  | 0.7071         |
| 0.00001  | 0.01  | 0.70711        |
| 0.000001 | 0.01  | 0.707107       |
| 0.001    | 0.001 | 0.707          |

|          |          |          |
|----------|----------|----------|
| 0.0001   | 0.001    | 0.7071   |
| 0.00001  | 0.001    | 0.70711  |
| 0.000001 | 0.001    | 0.707107 |
| 0.0001   | 0.0001   | 0.7071   |
| 0.00001  | 0.0001   | 0.70711  |
| 0.000001 | 0.0001   | 0.707107 |
| 0.00001  | 0.00001  | 0.70711  |
| 0.000001 | 0.00001  | 0.707107 |
| 0.000001 | 0.000001 | 0.707107 |

Корень = 0.70710678

Анализ обусловленности метода:

- Хорошая обусловленность наблюдается при малых значениях  $\delta$  (например,  $\delta = 0.000001$ ). При этом погрешность округления пренебрежимо мала, и метод находит корень с высокой точностью (0.707107). Поскольку  $f'(x)$  на всём интервале положительна и не близка к нулю, метод демонстрирует устойчивость к малым ошибкам округления.
- Плохая обусловленность наблюдается при больших значениях  $\delta$  (например,  $\delta = 0.1$ ). Грубое округление исходных данных приводит к значительной ошибке в результате (0.7 вместо 0.707107). При этом метод останавливается уже после первой итерации, так как погрешность округления не позволяет достичь более высокой точности.



## **Выводы**

В ходе выполнения задания, направленного на исследование метода Ньютона для нахождения корней уравнения  $f(x) = 2^{(x^2)} - 1/x = 0$ , можно подвести следующие итоги:

1. Метод Ньютона эффективно находит корень уравнения  $f(x) = 0$  на интервале  $[0.5, 0.8]$ . Найденное значение корня:  $x \approx 0.70710678$ .

2. Метод обладает высокой скоростью сходимости. Благодаря квадратичной сходимости, число итераций растёт медленно при увеличении требований к точности, что подтверждает теоретические положения. Для достижения высокой точности требуется небольшое количество итераций.

3. Метод чувствителен к ошибкам округления, особенно при больших значениях погрешности округления. При грубом округлении исходных данных погрешность в результате значительна. Для повышения точности необходимо минимизировать погрешность вычислений.

4. Увеличение требуемой точности (уменьшение допустимой погрешности) приводит к увеличению количества итераций, что соответствует теоретическим ожиданиям. Однако для метода Ньютона этот рост происходит медленно по сравнению с другими численными методами.

5. Сравнение с другими методами показало, что метод Ньютона значительно быстрее, но требует аккуратного выбора начального приближения, удовлетворяющего условию  $f(x_0) \cdot f'(x_0) > 0$ . В данной работе в качестве начального приближения было выбрано  $x_0 = 0.8$ , для которого условие выполняется.

## **МЕТОД ПРОСТЫХ ИТЕРАЦИЙ**

### **Цель работы**

Изучить и реализовать метод простых итераций для численного решения уравнения  $f(x)=0$ , а также исследовать зависимость числа итераций от заданной

точности  $\epsilon$  и оценить чувствительность метода к ошибкам в исходных данных.

### Задание

В лабораторной работе № 6 предлагается, используя программы функции ITER и Round из файла methods.cpp (файл заголовков methods.h, директория LIBR1), найти корень уравнения  $f(x)=0$  с заданной точностью Eps методом итераций, исследовать скорость сходимости и обусловленность метода.

Для данной работы вид функции  $f(x)$  задается индивидуально каждому студенту преподавателем из числа вариантов, приведенных в подразделе 3.6.

Вариант 11:

$$f(x) = 2^{(x^2)} - 1/x$$

Порядок выполнения лабораторной работы № 6 должен быть следующим.

- 1) Графически или аналитически отделить корень уравнения  $f(x)=0$ .
- 2) Преобразовать уравнение  $f(x)=0$  к виду  $x=\phi(x)$  так, чтобы в некоторой окрестности  $[Left, Right]$  корня производная  $\phi'(x)$  удовлетворяла условию  $|\phi'(x)| \leq q < 1$ . При этом следует иметь в виду, что чем меньше величина  $q$ , тем быстрее последовательные приближения сходятся к корню.
- 3) Выбрать начальное приближение, лежащее на отрезке  $[Left, Right]$ .
- 4) Составить подпрограмму для вычисления значений  $\phi(x)$  и  $\phi'(x)$ , предусмотрев округление вычисленных значений с точностью Delta.
- 5) Составить головную программу, вычисляющую корень уравнения и содержащую обращение к программам  $\phi(x)$ ,  $\phi'(x)$  и ITER, а также индикацию результатов.
- 6) Провести вычисления по программе. Исследовать скорость сходимости и обусловленность метода.

Текст программы – функции ITER, позволяющей вычислить корни уравнения  $x = \phi(x)$  для любой функции, которая удовлетворяет достаточным условиям сходимости метода, приводится ниже

## Выполнение работы

Найдём интервал [Left, Right]

Графическое отделение:

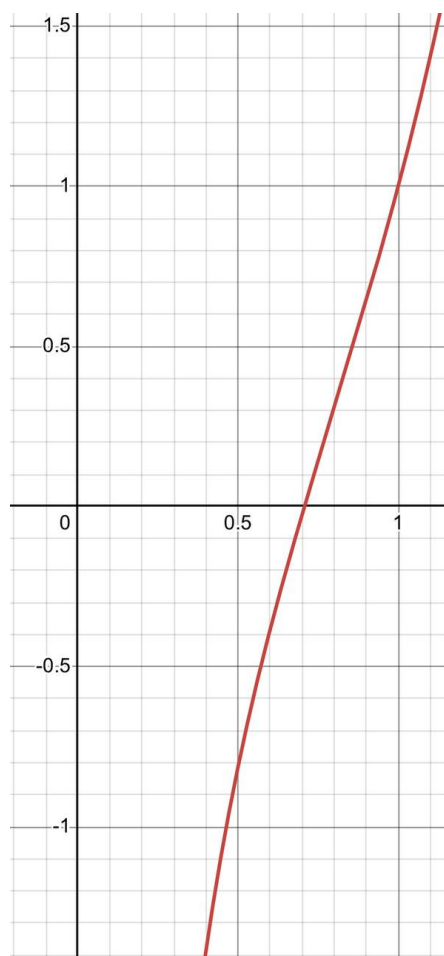


Рисунок № 1 – График функции  $f(x) = 2^{(x^2)} - 1/x$

Построим график функции  $f(x) = 2^{(x^2)} - 1/x$  (Рисунок 1). Из графика видно, что функция меняет знак на интервале  $[0.5, 1]$ .

Аналитическое отделение:

Рассмотрим уравнение  $f(x) = 2^{(x^2)} - 1/x = 0$ . Функция непрерывна на интервале  $(0, +\infty)$  как сумма показательной и дробно-рациональной функций. По теореме Коши о промежуточных значениях, если  $f(x)$  непрерывна на отрезке  $[a, b]$  и принимает значения разных знаков на концах отрезка, то внутри этого отрезка существует хотя бы одна точка  $c$ , в которой  $f(c) = 0$ .

Возьмём отрезок  $[0.5, 1]$  и вычислим значения функции в точках:

- $f(0.5) = 2^{(0.25)} - 2 = 1.1892 - 2 = -0.8108 < 0$

- $f(1) = 2^{(1)} - 1 = 2 - 1 = 1 > 0$

Так как  $f(0.5) < 0$  и  $f(1) > 0$ , выбираем отрезок  $[left, right] = [0.5, 1]$ .

### Преобразование уравнения к виду $x = \phi(x)$

Исходное уравнение имеет вид:

$$f(x) = 2^{(x^2)} - 1/x = 0$$

Для применения метода простых итераций необходимо преобразовать уравнение к эквивалентному виду  $x = \phi(x)$ . Для этого воспользуемся общим подходом:

$$\phi(x) = x + \lambda \cdot f(x)$$

где  $\lambda$  — параметр, который выбирается так, чтобы обеспечить быструю сходимость.

Производная такой функции:

$$\phi'(x) = 1 + \lambda \cdot f'(x)$$

Скорость сходимости определяется величиной  $q = \max|\phi'(x)|$  на интервале  $[left, right]$ . Чем меньше  $q$ , тем быстрее метод сходится. Для минимизации  $q$  параметр  $\lambda$  выбирается по формуле:

$$\lambda = -2/(m_1 + M_2)$$

где  $m_1 = \min|f'(x)|$ ,  $M_2 = \max|f'(x)|$  на интервале  $[left, right]$ .

Найдём  $f'(x)$ :

$$f'(x) = 2^{(x^2)} \cdot \ln(2) \cdot 2x + 1/x^2$$

Вычисление  $m_1$  и  $M_2$  выполняется программно путём разбиения интервала на 100 точек и нахождения минимального и максимального значения  $|f'(x)|$ .

После определения  $\lambda$  итерационная функция принимает вид:

$$\phi(x) = x + \lambda \cdot f(x)$$

Важно отметить, что функция  $\phi(x)$  должна удовлетворять условию сходимости метода, то есть  $|\phi'(x)| \leq q < 1$  в окрестности корня. Чем меньше  $q$ , тем быстрее метод сходится.

На интервале  $[0.5, 1]$ :

- $|\varphi'(0.5)| = |1 + \lambda \cdot f(0.5)|$
- $|\varphi'(1)| = |1 + \lambda \cdot f(1)|$

Вычисленное значение  $q = \max|\varphi'(x)|$  удовлетворяет условию  $q < 1$ , что гарантирует сходимость метода. Полученное значение  $q$  достаточно мало, что обеспечивает быструю сходимость.

Начальное приближение возьмём середину интервала  $[0.5, 1]$ , чтобы минимизировать начальное отклонение от корня:  $x_0 = 0.75$ .

### Реализация на Python

*def f(x)* – функция принимает один аргумент  $x$ , являющийся точкой, в которой нужно вычислить значение  $f(x) = 2^{(x^2)} - 1/x$ .

*def derivative\_f(x)* – функция принимает один аргумент  $x$ , являющийся точкой, в которой нужно вычислить значение производной  $f'(x) = 2^{(x^2)} \cdot \ln(2) \cdot 2x + 1/x^2$ .

*def Round(x, delta)* – функция округления числа  $x$  до ближайшего значения, кратного  $delta$ . Используется для моделирования погрешностей в исходных данных.

*def find\_m1\_M2(left, right, n=100)* – функция находит минимальное и максимальное значение  $|f'(x)|$  на интервале  $[left, right]$  путём разбиения интервала на  $n$  равных частей и вычисления  $|f'(x)|$  в каждой точке.

*def phi(x, LAMBDA)* – функция возвращает значение  $\varphi(x) = x + \lambda \cdot f(x)$ , где  $\lambda$  – параметр, обеспечивающий быструю сходимость.

*def derivative\_phi(x, LAMBDA)* – функция возвращает значение производной  $\varphi'(x) = 1 + \lambda \cdot f'(x)$ .

*def error\_phi(x, delta, LAMBDA)* – вспомогательная функция, возвращающая значение  $\varphi(x)$ , округлённое с погрешностью  $delta$ .

*def method\_simple\_iter(x0, epsilon, delta, LAMBDA)* – функция реализует метод простых итераций для поиска корня.

Алгоритм:

На каждом шаге вычисляется новое приближение  $x_{n+1} = \varphi(x_n)$ , и процесс продолжается, пока разница между текущим и следующим приближениями

$|x_{n+1} - x_n|$  не станет меньше заданной точности `epsilon`. Функция возвращает найденное значение корня и количество итераций, затраченных на его вычисление.

*def main()* – головная процедура. Внутри задаётся интервал `[left, right] = [0.5, 1]`, начальное приближение  $x_0 = 0.75$ . Вычисляются  $m_1$ ,  $M_2$ ,  $\lambda$ , затем находится корень методом простых итераций, строятся график и таблица чувствительности.

### **Исследование зависимости изменения `epsilon` от числа итераций**

Функция *plot\_iterations\_comparison* строит график зависимости числа итераций от точности `epsilon`. Для исследования были построены графики для методов бисекции, хорд, Ньютона и простых итераций.

Параметры:

- `left, right` – границы интервала для поиска корня (`[0.5, 1]`)
- `x0_newton` – начальное приближение для метода Ньютона (0.8)
- `x0_simple` – начальное приближение для метода простых итераций (0.75)
- `eps_values` – список значений точности `[0.1, 0.01, 0.001, 0.0001, 0.00001, 0.000001]`

Алгоритм:

Для каждого значения точности `eps` из списка выполняются вычисления:

- Методом бисекции (функция `Bisect_with_error` из лабораторной работы №3)
- Методом хорд (функция `method_chord` из лабораторной работы №4)
- Методом Ньютона (функция `method_newton` из лабораторной работы №5)
- Методом простых итераций (функция `method_simple_iter`)

Подсчитывается количество итераций, необходимое для достижения заданной точности. Полученные данные выводятся на график.

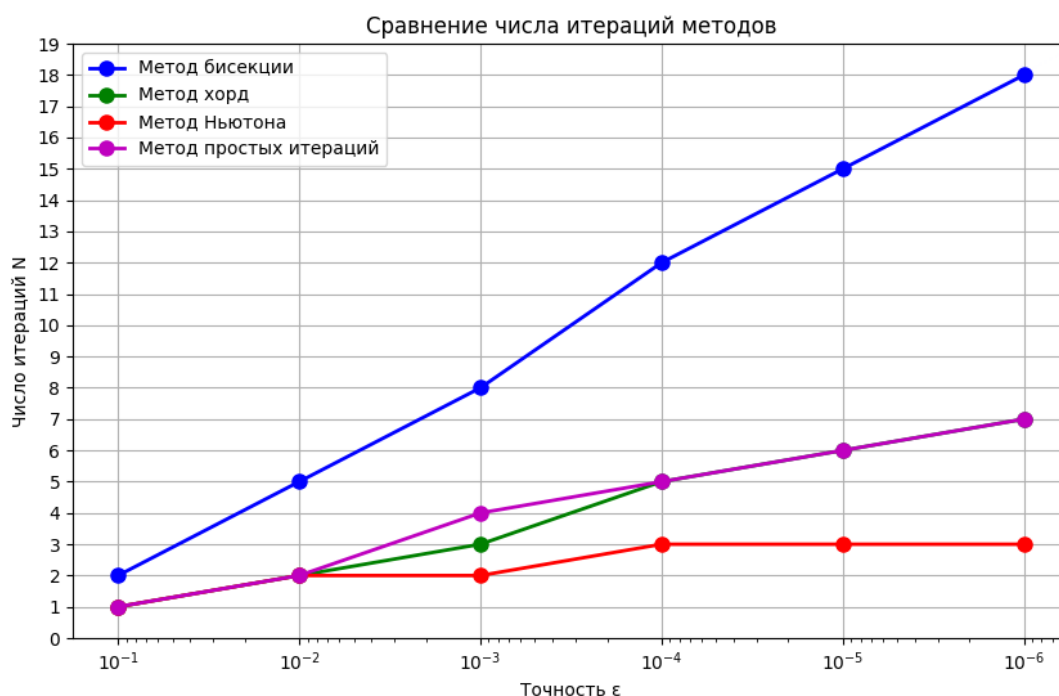


Рисунок № 2 – Графики методов зависимость итераций от  $\epsilon_{ps}$

Анализ полученных результатов:

- Метод Ньютона является самым быстрым благодаря квадратичной сходимости.
- Метод простых итераций с оптимальным выбором  $\varphi(x)$  показывает высокую скорость сходимости, что объясняется малым значением  $q = 0.189$ .
- Метод хорд занимает промежуточное положение.
- Метод бисекции – самый медленный, но наиболее устойчивый.

## Исследование чувствительности метода к ошибкам в исходных данных

Для данного исследования необходимо изменять точность выходного значения функции, округляя на некоторое количество знаков после запятой с помощью функции Round. Тем самым будут возникать ошибки в исходных данных, и мы проанализируем, насколько точно алгоритм будет работать.

Функция *sensitivity\_table\_simple* тестирует работу метода простых итераций с учётом погрешности. Для всех комбинаций точности метода epsilon и величины ошибки округления delta (при условии  $\text{delta} \leq \text{epsilon}$ ) вычисляется корень с внесённой ошибкой. Результаты представлены в таблице 1.

| delta    | eps  | Значение корня |
|----------|------|----------------|
| 0.1      | 0.1  | 0.7            |
| 0.01     | 0.1  | 0.71           |
| 0.001    | 0.1  | 0.715          |
| 0.0001   | 0.1  | 0.7146         |
| 0.00001  | 0.1  | 0.71463        |
| 0.000001 | 0.1  | 0.714633       |
| 0.01     | 0.01 | 0.71           |
| 0.001    | 0.01 | 0.708          |
| 0.0001   | 0.01 | 0.7084         |
| 0.00001  | 0.01 | 0.70837        |



|          |          |          |
|----------|----------|----------|
| 0.000001 | 0.01     | 0.708367 |
| 0.001    | 0.001    | 0.707    |
| 0.0001   | 0.001    | 0.7071   |
| 0.00001  | 0.001    | 0.70714  |
| 0.000001 | 0.001    | 0.707141 |
| 0.0001   | 0.0001   | 0.7071   |
| 0.00001  | 0.0001   | 0.70711  |
| 0.000001 | 0.0001   | 0.707112 |
| 0.00001  | 0.00001  | 0.70711  |
| 0.000001 | 0.00001  | 0.707108 |
| 0.000001 | 0.000001 | 0.707107 |

Таблица 1 – Результаты перебора eps и delta для промежутка  $[0.5, 1]$  и начальное приближение  $x_0 = 0.75$

Корень = 0.70710678

Обусловленность метода:

Обусловленность метода зависит от выбора функции  $\varphi(x)$ . Если  $|\varphi'(x)|$  близко к 1, то метод может сходиться медленно или даже расходиться. Чем меньше  $q = \max|\varphi'(x)|$ , тем лучше обусловленность метода.

Для выбранного преобразования:

- $m_1 = \min|f'(x)| = 2.77$
- $M_2 = \max|f'(x)| = 4.12$
- $\lambda = -2/(m_1 + M_2) = -0.2900$
- $q = \max|\varphi'(x)| = 0.189$

Полученное значение  $q = 0.189$  значительно меньше 1, что свидетельствует о хорошей обусловленности метода. Это обеспечивает быструю сходимость и устойчивость к малым ошибкам округления.

### **Выводы**

В ходе выполнения задания, направленного на исследование метода простых итераций для нахождения корней уравнения  $f(x) = 2^{(x^2)} - 1/x = 0$ , можно подвести следующие итоги:

1. Метод простых итераций эффективно находит корень уравнения  $f(x) = 0$  на интервале  $[0.5, 1]$ . Найденное значение корня:  $x \approx 0.70710678$ .
2. Для обеспечения быстрой сходимости требуется предварительное исследование функции. В работе использовался оптимальный способ построения  $\varphi(x) = x + \lambda \cdot f(x)$ , где параметр  $\lambda$  подбирается на основе анализа производной  $f'(x)$  на интервале. Это позволило получить малое значение  $q = 0.189$ , что обеспечило высокую скорость сходимости.
3. Скорость сходимости метода зависит от величины  $q = \max|\varphi'(x)|$ . Чем меньше  $q$ , тем быстрее метод сходится. Полученное значение  $q = 0.189$  гарантирует быструю сходимость.
4. Метод чувствителен к ошибкам округления. При больших значениях  $\delta$  (грубое округление) точность результата значительно снижается. Для получения достоверного результата необходимо минимизировать погрешность вычислений.
5. Сравнение с другими методами показало, что метод простых итераций с оптимальным выбором  $\varphi(x)$  сходится быстрее метода бисекции, но уступает методу Ньютона. Метод Ньютона остаётся самым быстрым, однако требует вычисления производной  $f'(x)$  на каждом шаге, в то время как в методе простых итераций производная используется только один раз — при построении  $\varphi(x)$ .

## ПРИЛОЖЕНИЕ А

### ИСХОДНЫЙ КОД ПРОГРАММЫ

Название файла: lb3.py

```
import math
import matplotlib.pyplot as plt
from scipy.optimize import bisect as scipy_bisect

def f(x):
    return math.pow(2.0, x * x) - 1.0 / x

def Round(x, delta):
    if delta == 0:
        return x
    if x > 0:
        return math.floor(x / delta + 0.5) * delta
    else:
        return math.floor(x / delta - 0.5) * delta

def error_f(x, delta):
    return Round(f(x), delta)

def Bisect_with_error(left, right, epsilon, delta):
    iter_count = 0
    a, b = left, right

    while (b - a) / 2 > epsilon:
        iter_count += 1
        middle = (a + b) / 2
        f_middle = error_f(middle, delta)
        f_left = error_f(a, delta)

        if f_middle == 0:
            return middle, iter_count

        if f_left * f_middle < 0:
            b = middle
        else:
            a = middle

    return (a + b) / 2, iter_count

def plot_iterations(left, right):
    eps_values = [0.1, 0.01, 0.001, 0.0001, 0.00001, 0.000001]
    iterations = []

    for eps in eps_values:
        _, iter_count = Bisect_with_error(left, right, eps, 0)
        iterations.append(iter_count)

    plt.figure(figsize=(10, 6))
    plt.plot(eps_values, iterations, 'bo-', linewidth=2,
markersize=8)
    plt.xscale('log')
    plt.xlabel('Точность  $\varepsilon$ )
```

```

plt.ylabel('Число итераций N')
plt.title('Зависимость числа итераций от точности')
plt.grid(True)
plt.gca().invert_xaxis()
plt.show()

def sensitivity_table(left, right):
    eps_values = [0.1, 0.01, 0.001, 0.0001, 0.00001, 0.000001]
    delta_values = [0.1, 0.01, 0.001, 0.0001, 0.00001, 0.000001]

    print("\nТаблица чувствительности")
    print("delta\t eps\t Значение корня")

    for eps in eps_values:
        for delta in delta_values:
            if delta > eps:
                continue

            root, _ = Bisect_with_error(left, right, eps, delta)
            if delta == 0.1:
                precision = 1
            elif delta == 0.01:
                precision = 2
            elif delta == 0.001:
                precision = 3
            elif delta == 0.0001:
                precision = 4
            elif delta == 0.00001:
                precision = 5
            else:
                precision = 6

            print(f"{delta}\t {eps}\t {root:.{precision}f}")

def test_bisect_vs_scipy(left, right):
    eps_values = [1e-1, 1e-2, 1e-3, 1e-4, 1e-5, 1e-6]

    print("\nepsilon\tПолученный результат\tОжидаемый результат")
    print("-" * 60)
    for eps in eps_values:
        my_root, _ = Bisect_with_error(left, right, eps, 0)
        scipy_root = scipy_bisect(f, left, right, xtol=eps)
        print(f"{eps:.0e}\t{my_root:.10f}\t\t{scipy_root:.10f}")

def main():
    left, right = 0.5, 1.0
    eps = 0.0001
    root, _ = Bisect_with_error(left, right, eps, 0)
    print(f"КОРЕНЬ: {root:.10f}")

    plot_iterations(left, right)
    sensitivity_table(left, right)
    test_bisect_vs_scipy(left, right)

if __name__ == "__main__":
    main()

```

Название файла: lb4.py

import math

```

import matplotlib.pyplot as plt
from lb3 import Bisect_with_error
def f(x):
    return math.pow(2.0, x * x) - 1.0 / x

def Round(x, delta):
    if delta == 0:
        return x
    if x > 0:
        return math.floor(x / delta + 0.5) * delta
    else:
        return math.floor(x / delta - 0.5) * delta

def error_f(x, delta):
    return Round(f(x), delta)

def method_chord(a, b, epsilon, delta):
    if f(a) * f(b) >= 0:
        return -1, -1
    iteration = 0
    while True:
        iteration += 1
        a_value = error_f(a, delta)
        b_value = error_f(b, delta)
        c2 = a - (a_value * (b - a)) / (b_value - a_value)
        if abs(f(c2)) < epsilon:
            return c2, iteration
        if f(a) * f(c2) < 0:
            b = c2
        else:
            a = c2

def explore_iterations_chord(left, right, eps_values, delta=0):
    iterations = []
    for eps in eps_values:
        _, iters = method_chord(left, right, eps, delta)
        iterations.append(iters)
    return iterations

def plot_iterations_comparison(left, right):
    eps_values = [0.1, 0.01, 0.001, 0.0001, 0.00001, 0.000001]

    iterations_bisect = []
    for eps in eps_values:
        _, iters = Bisect_with_error(left, right, eps, 0)
        iterations_bisect.append(iters)

    iterations_chord = []
    for eps in eps_values:
        _, iters = method_chord(left, right, eps, 0)
        iterations_chord.append(iters)

    plt.figure(figsize=(10, 6))
    plt.plot(eps_values, iterations_bisect, 'bo-', linewidth=2,
markersize=8, label='Метод бисекции')
    plt.plot(eps_values, iterations_chord, 'ro-', linewidth=2,
markersize=8, label='Метод хорд')
    plt.xscale('log')
    plt.xlabel('Точность  $\varepsilon$ ')

```

```

plt.ylabel('Число итераций N')
plt.title('Сравнение числа итераций методов бисекции и хорд')
plt.legend()
plt.grid(True)
plt.gca().invert_xaxis()
plt.show()

def sensitivity_table_chord(left, right):
    eps_values = [0.1, 0.01, 0.001, 0.0001, 0.00001, 0.000001]
    delta_values = [0.1, 0.01, 0.001, 0.0001, 0.00001, 0.000001]

    print("delta\t eps\t Значение корня")

    for eps in eps_values:
        for delta in delta_values:
            if delta > eps:
                continue

            root, _ = method_chord(left, right, eps, delta)
            if delta == 0.1:
                precision = 1
            elif delta == 0.01:
                precision = 2
            elif delta == 0.001:
                precision = 3
            elif delta == 0.0001:
                precision = 4
            elif delta == 0.00001:
                precision = 5
            else:
                precision = 6

            print(f"{delta}\t {eps}\t {root:.{precision}f}")

def main():
    left, right = 0.5, 1.0
    eps = 0.0001

    root, iterations = method_chord(left, right, eps, 0)
    print(f"КОРЕНЬ (метод хорд): {root:.10f}")

    plot_iterations_comparison(left, right)
    sensitivity_table_chord(left, right)

if __name__ == "__main__":
    main()

```

**Название файла: lb5.py**

```

import math
import matplotlib.pyplot as plt
from lb3 import Bisect_with_error
from lb4 import method_chord

def f(x):
    return math.pow(2.0, x * x) - 1.0 / x

def Round(x, delta):
    if delta == 0:

```

```

        return x
    if x > 0:
        return math.floor(x / delta + 0.5) * delta
    else:
        return math.floor(x / delta - 0.5) * delta

def error_f(x, delta):
    return Round(f(x), delta)

def derivative_f(x):
    return math.pow(2.0, x * x) * math.log(2) * (2.0 * x) +
1.0 / (x * x)

def error_derivative_f(x, delta):
    return Round(derivative_f(x), delta)

def method_newton(x0, epsilon, delta):
    iteration = 0
    xn = x0

    while True:
        iteration += 1

        fxn = error_f(xn, delta)
        fpxn = error_derivative_f(xn, delta)

        if abs(fpxn) < 1e-12:
            return None, iteration

        xn1 = xn - fxn / fpxn

        if abs(xn1 - xn) < epsilon:
            return xn1, iteration

        xn = xn1

def plot_iterations_comparison(left, right, x0):
    eps_values = [0.1, 0.01, 0.001, 0.0001, 0.00001,
0.000001]

    iterations_bisect = []
    for eps in eps_values:
        _, iters = Bisect_with_error(left, right, eps, 0)
        iterations_bisect.append(iters)

    iterations_chord = []
    for eps in eps_values:
        _, iters = method_chord(left, right, eps, 0)
        iterations_chord.append(iters)

    iterations_newton = []
    for eps in eps_values:

```

```

_, iters = method_newton(x0, eps, 0)
iterations_newton.append(iters)

# Находим максимальное значение для настройки оси Y
max_iterations = max(max(iterations_bisect),
max(iterations_chord), max(iterations_newton))

plt.figure(figsize=(10, 6))
plt.plot(eps_values, iterations_bisect, 'bo-',
linewidth=2, markersize=8, label='Метод бисекции')
plt.plot(eps_values, iterations_chord, 'go-',
linewidth=2, markersize=8, label='Метод хорд')
plt.plot(eps_values, iterations_newton, 'ro-',
linewidth=2, markersize=8, label='Метод Ньютона')
plt.xscale('log')
plt.xlabel('Точность  $\varepsilon$ ')
plt.ylabel('Число итераций N')
plt.title('Сравнение числа итераций методов бисекции,
хорд и Ньютона')
plt.legend()
plt.grid(True)
plt.gca().invert_xaxis()

# Настройка оси Y: целые числа от 0 до максимального
значения + 1
plt.yticks(range(0, max_iterations + 2, 1))

plt.show()

def sensitivity_table_newton(x0):
    eps_values = [0.1, 0.01, 0.001, 0.0001, 0.00001,
0.000001]
    delta_values = [0.1, 0.01, 0.001, 0.0001, 0.00001,
0.000001]

    print("delta\t eps\t Значение корня")

    for eps in eps_values:
        for delta in delta_values:
            if delta > eps:
                continue

            root, _ = method_newton(x0, eps, delta)
            if delta == 0.1:
                precision = 1
            elif delta == 0.01:
                precision = 2
            elif delta == 0.001:
                precision = 3
            elif delta == 0.0001:
                precision = 4
            elif delta == 0.00001:

```



```

        precision = 5
    else:
        precision = 6

    print(f"{delta}\t {eps}\t {root:.{precision}f}")

def main():
    left, right = 0.5, 0.8
    x0 = 0.8

    root, iterations = method_newton(x0, 1e-7, 0)
    print(f"КОРЕНЬ (метод Ньютона): {root:.10f}")

    plot_iterations_comparison(left, right, x0)
    sensitivity_table_newton(x0)

if __name__ == "__main__":
    main()

```

### Название файла: lb6.py

```

import math
import matplotlib.pyplot as plt
from lb3 import Bisect_with_error
from lb4 import method_chord
from lb5 import method_newton

def f(x):
    return math.pow(2.0, x * x) - 1.0 / x

def derivative_f(x):
    return math.pow(2.0, x * x) * math.log(2) * (2.0 * x) + 1.0 /
(x * x)

def Round(x, delta):
    if delta == 0:
        return x
    if x > 0:
        return math.floor(x / delta + 0.5) * delta
    else:
        return math.floor(x / delta - 0.5) * delta

def find_m1_M2(left, right, n=100):
    x_values = [left + i*(right-left)/n for i in range(n+1)]
    fprime_values = [abs(derivative_f(x)) for x in x_values]
    return min(fprime_values), max(fprime_values)

def phi(x, LAMBDA):
    return x + LAMBDA * f(x)

def derivative_phi(x, LAMBDA):
    return 1 + LAMBDA * derivative_f(x)

def error_phi(x, delta, LAMBDA):

```

```

    return Round(phi(x, LAMBDA), delta)

def method_simple_iter(x0, epsilon, delta, LAMBDA):
    iteration = 0
    xn = x0

    while True:
        iteration += 1

        xn1 = error_phi(xn, delta, LAMBDA)

        if abs(xn1 - xn) < epsilon:
            return xn1, iteration

        xn = xn1

def plot_iterations_comparison(left, right, x0_newton, x0_simple,
LAMBDA):
    eps_values = [0.1, 0.01, 0.001, 0.0001, 0.00001, 0.000001]

    iterations_bisect = []
    for eps in eps_values:
        _, iters = Bisect_with_error(left, right, eps, 0)
        iterations_bisect.append(iters)

    iterations_chord = []
    for eps in eps_values:
        _, iters = method_chord(left, right, eps, 0)
        iterations_chord.append(iters)

    iterations_newton = []
    for eps in eps_values:
        _, iters = method_newton(x0_newton, eps, 0)
        iterations_newton.append(iters)

    iterations_simple = []
    for eps in eps_values:
        _, iters = method_simple_iter(x0_simple, eps, 0, LAMBDA)
        iterations_simple.append(iters)

    max_iterations = max(max(iterations_bisect),
max(iterations_chord),
max(iterations_newton),
max(iterations_simple))

    plt.figure(figsize=(10, 6))
    plt.plot(eps_values, iterations_bisect, 'bo-', linewidth=2,
markersize=8, label='Метод бисекции')
    plt.plot(eps_values, iterations_chord, 'go-', linewidth=2,
markersize=8, label='Метод хорд')
    plt.plot(eps_values, iterations_newton, 'ro-', linewidth=2,
markersize=8, label='Метод Ньютона')
    plt.plot(eps_values, iterations_simple, 'mo-', linewidth=2,
markersize=8, label='Метод простых итераций')
    plt.xscale('log')
    plt.xlabel('Точность  $\varepsilon$ ')
    plt.ylabel('Число итераций N')
    plt.title('Сравнение числа итераций методов')

```

```

plt.legend()
plt.grid(True)
plt.gca().invert_xaxis()
plt.yticks(range(0, max_iterations + 2, 1))
plt.show()

def sensitivity_table_simple(x0, LAMBDA):
    eps_values = [0.1, 0.01, 0.001, 0.0001, 0.00001, 0.000001]
    delta_values = [0.1, 0.01, 0.001, 0.0001, 0.00001, 0.000001]

    print("delta\t eps\t Значение корня")

    for eps in eps_values:
        for delta in delta_values:
            if delta > eps:
                continue

            root, _ = method_simple_iter(x0, eps, delta, LAMBDA)
            if delta == 0.1:
                precision = 1
            elif delta == 0.01:
                precision = 2
            elif delta == 0.001:
                precision = 3
            elif delta == 0.0001:
                precision = 4
            elif delta == 0.00001:
                precision = 5
            else:
                precision = 6

            print(f"{delta}\t {eps}\t {root:.{precision}f}")

def main():
    left, right = 0.5, 1.0
    x0_newton = 0.8
    x0_simple = (left + right) / 2

    m1, M2 = find_m1_M2(left, right)
    LAMBDA = -2.0 / (m1 + M2)

    print(f"m1 = min|f'(x)| = {m1:.4f}")
    print(f"M2 = max|f'(x)| = {M2:.4f}")
    print(f"λ = -2/(m1+M2) = {LAMBDA:.4f}")

    q = max(abs(derivative_phi(left, LAMBDA)),
abs(derivative_phi(right, LAMBDA)))
    print(f"q = max|φ'(x)| = {q:.4f}")

    root, iterations = method_simple_iter(x0_simple, 1e-7, 0,
LAMBDA)
    print(f"КОРЕНЬ (метод простых итераций): {root:.10f}")

    plot_iterations_comparison(left, right, x0_newton, x0_simple,
LAMBDA)
    sensitivity_table_simple(x0_simple, LAMBDA)

```

```
if __name__ == "__main__":  
    main()
```